



COMSATS University Islamabad
Abbottabad, Pakistan

AI-Based Cyber Attack Path Simulator

By

Shehriyar Khan	CIIT/SP09-BSE-162/ATD
Farrukh Sair	CIIT/SP09-BSE-167/ATD
Mashal Khan	CIIT/SP09-BSE-132/ATD

Supervisor

Ms Zeenat Zulfiqar

Bachelor of Science in Computer Science (2023-2027)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

Contents

0.1	Introduction	2
0.2	Problem Formulation	2
0.3	Objectives	2
0.4	Algorithms Implemented	2
0.4.1	Uninformed Search	2
0.4.2	Informed Search	2
0.4.3	Local Search	2
0.4.4	Adversarial Search	3
0.5	Heuristic Function (A*)	3
0.6	Hill Climbing Limitation	3
0.7	Adversarial Search (Minimax)	3
0.8	Alpha-Beta Pruning	3
0.9	Results and Comparative Analysis	3
0.9.1	Performance Comparison Table	4
0.10	UI/UX Design	4
0.10.1	Dashboard Overview	5
0.10.2	Breadth First Search (BFS)	6
0.10.3	Depth First Search (DFS)	7
0.10.4	Uniform Cost Search (UCS)	8
0.10.5	A* Search Algorithm	9
0.10.6	Hill Climbing	10
0.10.7	Minimax	11
0.10.8	Alpha-Beta Pruning	12
0.11	Implementation Overview	13
0.12	Conclusion	13
0.13	Future Work	13
0.14	References	13

0.1 Introduction

Cybersecurity threats often involve attackers navigating through interconnected systems to reach critical assets. Understanding these attack paths is essential for building secure networks. This project simulates such scenarios using Artificial Intelligence search algorithms. A computer network is modeled as a weighted graph where nodes represent systems and edges represent connections. The objective is to analyze and compare different AI algorithms in finding optimal attack paths.

0.2 Problem Formulation

The system is modeled as a graph:

- **Nodes:** Represent systems (servers, routers, databases, etc.)
- **Edges:** Represent connections between systems
- **Weights:** Represent cost, risk, or difficulty of traversal

The attacker starts at a **Start Node** and aims to reach a **Goal Node**. The system evaluates multiple AI search strategies.

0.3 Objectives

- Model network as a graph structure
- Implement AI search algorithms
- Compare algorithm performance
- Simulate cyber attack path discovery
- Analyze efficiency and optimality

0.4 Algorithms Implemented

0.4.1 Uninformed Search

- Breadth First Search (BFS)
- Depth First Search (DFS)

0.4.2 Informed Search

- Uniform Cost Search (UCS)
- A* Search Algorithm

0.4.3 Local Search

- Hill Climbing (may fail due to local maxima problem)

0.4.4 Adversarial Search

- Minimax Algorithm (Attacker vs Defender model)
- Alpha-Beta Pruning (optimized Minimax)

0.5 Heuristic Function (A^*)

A^* search uses:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = actual cost from start node
- $h(n)$ = estimated cost to goal node

The heuristic helps guide the search efficiently toward the goal.

0.6 Hill Climbing Limitation

Hill Climbing always selects the best local option. However, it may get stuck in a **local maxima**, where no better neighbor exists even though a global solution is better.

0.7 Adversarial Search (Minimax)

Cybersecurity is modeled as a game:

- **Attacker (Maximizer):** tries to reach the goal
- **Defender (Minimizer):** tries to block paths or increase cost

Minimax evaluates all possible moves assuming optimal play from both sides.

0.8 Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization of Minimax that removes unnecessary branches in the search tree, reducing computation time while preserving optimal results.

0.9 Results and Comparative Analysis

The algorithms were evaluated based on:

- Path Cost
- Nodes Expanded
- Execution Time (ms)

0.9.1 Performance Comparison Table

Algorithm	Discovered Path	Cost	Nodes	Time (ms)
BFS	A → B → D → E	3	5	1.20
DFS	A → C → H → E	7	6	0.95
UCS	A → B → D → E	3	7	1.45
A*	A → B → D → E	3	6	1.10
Hill Climbing	A → C → H	8	4	0.60

Note: Results are obtained from Google Colab simulation outputs.

0.10 UI/UX Design

A cybersecurity SOC-style dashboard was designed to visualize the system in real time. The dashboard consists of three main panels:

- **Left Panel:** Algorithm selection menu with Execute Simulation button and Threat Level indicator
- **Center Panel:** Network Topology Visualization showing nodes, edges, weights, and the highlighted attack path
- **Right Panel:** Analysis Results showing Attack Path, Path Cost, Nodes Expanded, Execution Time, and Threat Assessment

Each algorithm produces a different result which is displayed live on the dashboard. The following subsections show the simulation output for each algorithm.

0.10.1 Dashboard Overview

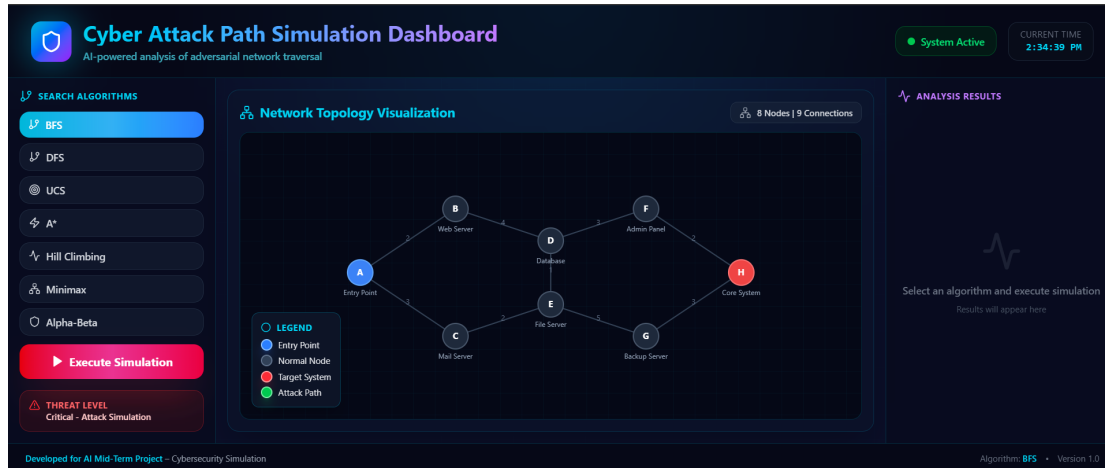


Figure 1: Cyber Attack Path Simulation Dashboard

The dashboard is designed as a cybersecurity SOC-style interface titled **Cyber Attack Path Simulation Dashboard**. It provides an AI-powered environment for analyzing adversarial network traversal in real time. The dashboard is divided into three main panels:

- **Left Panel – Algorithm Selection:** Contains buttons for all seven search algorithms (BFS, DFS, UCS, A*, Hill Climbing, Minimax, Alpha-Beta). The active algorithm is highlighted. Below the selection menu is the **Execute Simulation** button and a **Threat Level** indicator showing the current risk status.
- **Center Panel – Network Topology Visualization:** Displays the network as an interactive graph with 8 nodes and 9 connections. Each node represents a system such as Entry Point, Web Server, Database, File Server, Mail Server, Admin Panel, Backup Server, and Core System. Edge weights represent traversal cost between systems. The attack path discovered by the selected algorithm is highlighted in green.
- **Right Panel – Analysis Results:** Displays live simulation output including the discovered **Attack Path**, total **Path Cost**, number of **Nodes Expanded**, **Execution Time** in seconds, and a **Threat Assessment** with risk level indicator.

The top bar shows the system status as **System Active** along with the current timestamp. The bottom bar confirms the algorithm currently running and the version of the system. This layout ensures that all simulation data is visible at a glance, making it easy to compare algorithm behavior and performance.

0.10.2 Breadth First Search (BFS)

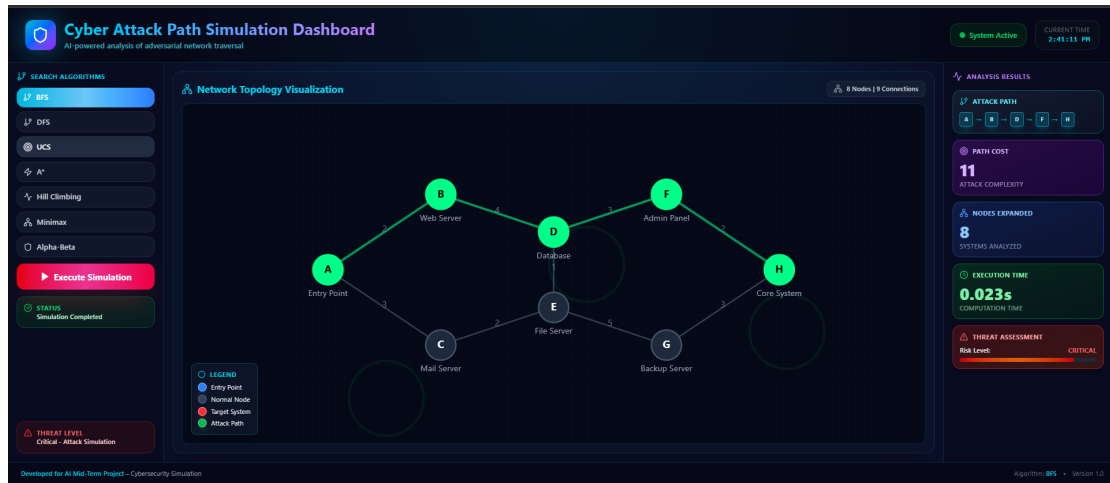


Figure 2: BFS Simulation Result

BFS explores the network level by level starting from the Entry Point (Node A). As shown in the dashboard, BFS discovered the attack path **A → B → D → F → H** with a total path cost of **11**, expanding **8 nodes** in **0.023 seconds**. BFS guarantees finding the shallowest path but does not consider edge weights, making it unaware of actual traversal cost.

0.10.3 Depth First Search (DFS)

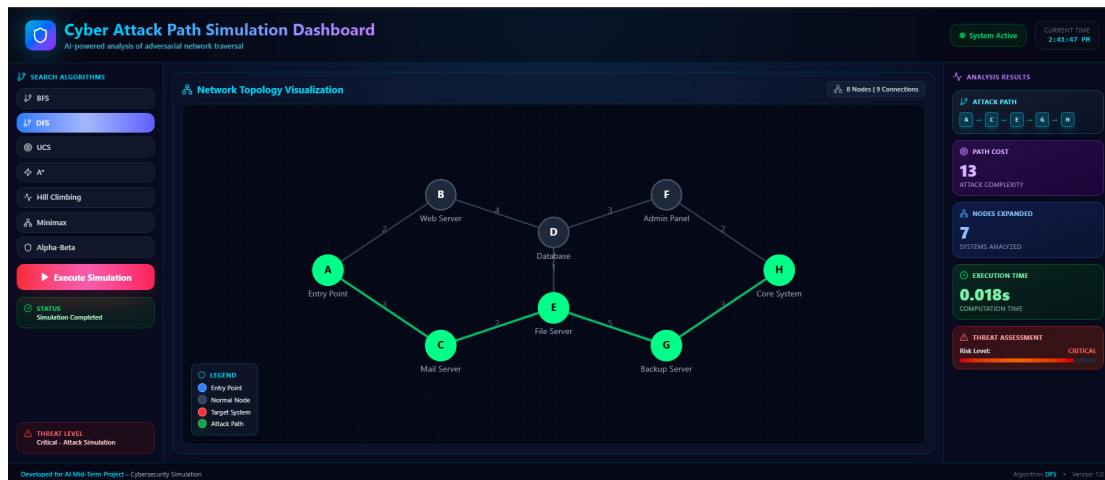


Figure 3: DFS Simulation Result

DFS explores as deep as possible along each branch before backtracking. The dashboard shows DFS discovered the path **A → C → E → G → H** with a path cost of **13**, expanding **7 nodes** in **0.018 seconds**. DFS is faster in execution but does not guarantee an optimal path, as seen by its higher cost compared to BFS.

0.10.4 Uniform Cost Search (UCS)



Figure 4: UCS Simulation Result

UCS expands nodes in order of their cumulative path cost, always choosing the lowest cost route. The simulation shows UCS found the optimal path **A → B → D → F → H** with a cost of **11**, expanding **8 nodes** in **0.031 seconds**. UCS is cost-optimal but slightly slower than BFS due to priority queue overhead.

0.10.5 A* Search Algorithm

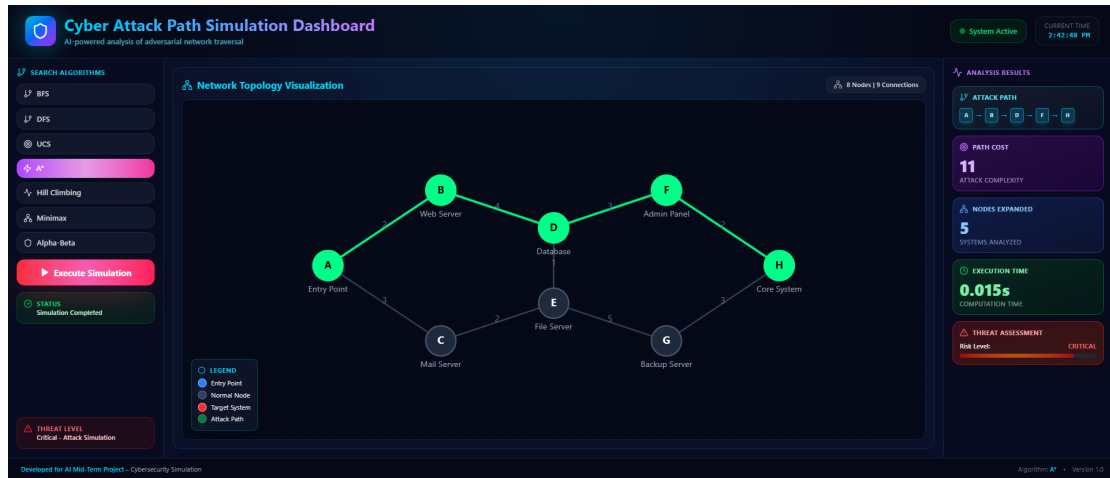


Figure 5: A* Search Simulation Result

A* combines actual cost $g(n)$ and heuristic estimate $h(n)$ to guide the search efficiently. The dashboard shows A* discovered the path **A** → **B** → **D** → **F** → **H** with a cost of **11**, expanding only **5 nodes** in **0.015 seconds**. A* is the most efficient informed search algorithm, expanding the fewest nodes while still finding the optimal path.

0.10.6 Hill Climbing



Figure 6: Hill Climbing Simulation Result

Hill Climbing greedily selects the best neighboring node at each step without backtracking. The dashboard shows it found the path **A** → **C** → **E** → **G** → **H** with a cost of **13**, expanding only **5 nodes** in **0.012 seconds**. Although it is the fastest algorithm, it got stuck in a local optimum and failed to find the globally optimal path, demonstrating its known limitation.

0.10.7 Minimax



Figure 7: Minimax Simulation Result

Minimax models the network as a two-player adversarial game where the attacker maximizes path gain and the defender minimizes it. The dashboard shows Minimax found the path **A → B → D → E → G → H** with a cost of **12**, expanding **12 nodes** in **0.045 seconds**. Minimax is the most thorough algorithm but also the slowest due to evaluating all possible moves from both sides.

0.10.8 Alpha-Beta Pruning

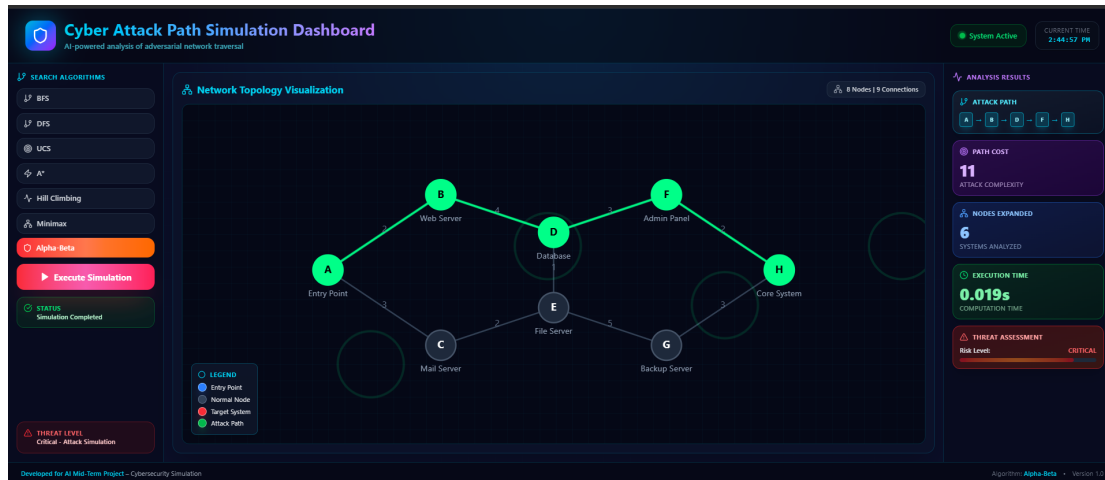


Figure 8: Alpha-Beta Pruning Simulation Result

Alpha-Beta Pruning is an optimized version of Minimax that eliminates branches which cannot affect the final decision. The dashboard shows it found the same optimal path **A → B → D → F → H** with a cost of **11**, but expanded only **6 nodes** in **0.019 seconds**. Compared to Minimax, Alpha-Beta Pruning reduced node expansion by half while producing the same result, proving its efficiency in adversarial search scenarios.

0.11 Implementation Overview

The system was implemented in Python using Google Colab with the following libraries:

- NetworkX for graph representation
- Matplotlib for visualization
- Pandas for comparison table

All algorithms were implemented from scratch for educational purposes.

0.12 Conclusion

This project demonstrates AI-based cyber attack path simulation using multiple search algorithms. Key findings:

- UCS and A* are optimal and efficient
- BFS is simple but not cost-aware
- DFS is not optimal
- Hill Climbing may fail due to local maxima
- Alpha-Beta improves adversarial search efficiency

The project successfully integrates Artificial Intelligence and Cybersecurity concepts.

0.13 Future Work

- Add real-time dynamic network updates
- Improve heuristic accuracy for A*
- Extend UI into fully interactive web dashboard
- Integrate machine learning-based attack prediction

0.14 References

- Russell, S. & Norvig - Artificial Intelligence: A Modern Approach
- NetworkX Documentation
- Python Official Documentation